Athel — 05 Spec, Blast Radius and Planning

Nov 1, 2026 · @Vaibhav Shukla

Before any code is written, Athel turns the PRD into traceable specs, measures what the change can break, asks only the questions that matter, and produces a plan a human approves.

Purpose, scope and user stories

This module decides **what** to build and **in what order**. It covers features F-07 (blast radius), F-08 (PRD intake and spec), F-09 (decision cards), F-10 (change plan) and F-21 (brownfield onboarding). It writes no code; its outputs are files the build loop consumes.

User stories

ID	As a	I want	So that
US-SPEC-01	Feature owner	Athel to find gaps and contradictions in my PRD before work starts	we don't build the wrong thing
US-SPEC-02	Reviewer	every code change traced to a PRD line and acceptance criterion	I can check intent, not just syntax
US-SPEC-03	Developer	Athel to learn an existing repo's conventions before changing it	its code looks native to the repo
US-SPEC-04	Developer	a blast radius report before any code, with real traffic numbers and a risk level	I can stop a risky change early
US-SPEC-05	Developer	the blast radius re-checked against the actual diff before each PR	late scope changes are caught
US-SPEC-06	Feature owner	only real questions, each with options and a recommendation	I answer in seconds, not hours
US-SPEC-07	Feature owner	to see what Athel will assume if I don't answer	the work isn't stuck on me
US-SPEC-08	Developer	an ordered, approvable plan across repos	I know exactly what will happen before it happens

Functional requirements

ID	Requirement	Acceptance criterion
FR-SPEC-001	PRD intake checks for: goal, actors, scope and out-of-scope, functional requirements, acceptance criteria, non-functional needs, constraints, and dependencies; missing ones are listed.	Fixture PRD without acceptance criteria yields a gap list naming it.
FR-SPEC-002	Intake detects contradictions and ambiguity ("fast", "secure", "etc.") and turns material ones into decision cards.	Fixture with two conflicting limits produces one card.
FR-SPEC-003	Each PRD statement gets a stable ID (`PRD-3.2`); every spec requirement cites at least one.	Traceability check: 100% of spec requirements cite a PRD ID.
FR-SPEC-004	Specs are written per service: requirements (EARS style), design notes, interfaces changed, data changes, acceptance criteria as testable examples.	Spec validates against `schemas/spec@1` .
FR-SPEC-005	Every acceptance criterion is testable: it names input, action and observable result.	Lint flags criteria without an observable result.
FR-SPEC-006	For an existing repo, onboarding derives conventions (structure, naming, error handling, logging, test style, lint and format config) before any spec is written.	`conventions.md` exists and is cited by the developer agent.
FR-BLAST-001	A blast radius report is produced after the spec and before plan approval, and again from the real diff before each PR.	Both reports exist for every change that reached PR.
FR-BLAST-002	The report covers direct code, contract consumers, data, infrastructure and config, people (owners and active work), and runtime (traffic, health).	Report validates against `schemas/blast@1` with all six sections.

ID	Requirement	Acceptance criterion
FR-BLAST-003	Each affected consumer includes evidence and observed traffic (calls per minute or events per minute, environment, window).	No consumer entry lacks evidence.
FR-BLAST-004	A risk level (LOW, MEDIUM, HIGH, CRITICAL) is computed by the documented rules, with the rules that fired listed.	Same input always gives same level (deterministic rules in the helper CLI).
FR-BLAST-005	If the pre-PR report finds impact not in the approved plan, the PR is blocked and the plan must be re-approved.	Fixture with an extra consumer blocks the PR.
FR-DEC-001	Decision cards are raised only when the ask rules (section 6) say so.	Card log shows the rule that triggered each card.
FR-DEC-002	Each card has: question, context, 2 to 4 options with consequences, a recommendation with reason, the default if unanswered, and what is blocked meanwhile.	Card validates against `schemas/decision@1`.
FR-DEC-003	Cards are batched: all open questions for a step are presented together.	Never more than one interruption per step unless a new blocker appears.
FR-DEC-004	Answers are recorded with who answered and when, and cited by later work.	Code citing a decision links its ID.
FR-PLAN-001	The plan is a task graph: each task has repo, paths or resources, verbs, acceptance criteria covered, dependencies, size estimate, and gates.	Plan validates against `schemas/plan@1`; graph is acyclic.
FR-PLAN-002	Every acceptance criterion is covered by at least one task, and every task covers at least one criterion.	Coverage check passes.
FR-PLAN-003	Multi-repo plans follow the expand and contract ordering (doc 04).	Ordering lint passes.

ID	Requirement	Acceptance criterion
FR-PLAN-004	The plan needs explicit human approval; approval is bound to the plan's hash.	Edited plan after approval returns state to `planned`.

Edge cases

Edge case	Required behaviour
PRD is a one-liner ("add dark mode")	Intake produces a draft spec with explicit assumptions and one batched card to confirm them; never silently invents scope
PRD is huge (50+ pages) or covers many features	Split into several changes, each with its own ID and plan; a parent record links them
PRD conflicts with existing behaviour in code	Flag as a decision card: change behaviour, keep it, or adapt the requirement
PRD references a service that doesn't exist or was renamed	Scout resolves aliases; if unresolved, card
The right service to change is ambiguous (data duplicated in two services)	Blast radius shows both with evidence; card recommends the owner of the write path
Change is purely internal (refactor, no contract change)	Blast radius still runs; consumers section states "no contract change detected" with the evidence used
No runtime data available (new service, no metrics yet)	Report says so; risk level cannot be LOW without runtime evidence for existing consumers
User answers a card with something outside the options	Accept free text; architect agent re-checks it for conflicts before use
User never answers	Keep working on every task the card does not block and send one reminder after 2 hours. A card marked `default_safe: true` applies its default after 4 hours, is logged, and is shown again at plan approval so it can be reversed; every other card stays blocked until answered

Edge case	Required behaviour
Plan approval with edits from the human	Human edits are validated (schema, coverage, ordering); approval binds to the edited plan's hash
PRD changes mid-build	New PRD version; spec diff; tasks affected are re-planned; unaffected completed tasks keep their state

PRD intake, spec format and brownfield onboarding

The architect agent writes specs from the PRD, the investigation and the repo's conventions; the helper CLI checks structure and traceability so specs cannot drift from the PRD unnoticed.

Intake steps (`/athel:start` , then `/athel:spec`)

1. Copy the PRD to `prd/v1.md` ; the helper CLI assigns IDs to each heading and requirement-like sentence (`PRD-3.2`).
2. The architect runs the completeness checklist (FR-SPEC-001) and an ambiguity scan; results go to `prd/review-v1.md` .
3. The scout investigates the services named or implied (doc 03).
4. Material gaps become batched decision cards; non-material gaps become stated assumptions.
5. Specs are written per service; the helper CLI checks schema and traceability.

Spec format (`spec/<service>.md`)

```
# Spec: user-service · CHG-042

## Requirements
- REQ-1 (PRD-2.1): When a signed-in customer submits a new phone number,
  the user-service shall validate it as E.164 and store it. [Decision D-1]
- REQ-2 (PRD-2.3): When the phone number changes, the user-service shall
  publish UserUpdated v2 with the new number within 1 second.

## Interfaces changed
- PUT /v1/users/{id}/phone (new) — request, response, errors
- Event UserUpdated v2 (additive: phone_number) — v1 unchanged

## Data changes
- users.phone_number (new column, nullable), backfill from phone
```

```
## Acceptance criteria
- AC-1 (REQ-1): Given a customer with phone +919800000000,
  when they submit +919811111111, then GET returns +919811111111.
- AC-2 (REQ-1): Given input "98111 11111", then 400 with code PHONE_INVALID.

## Out of scope
- Removing the old `phone` field (follow-up change)
```

Requirements use EARS patterns ("When …, the … shall …", "If …, then the … shall …") so each is unambiguous and testable.

Traceability: PRD ID → REQ → AC → task → commit trailer → test name. The helper CLI produces `traceability.md` and fails the plan step if any link is missing.

Brownfield onboarding (first change on a repo, refreshed when stale)

- Read build files, lint and format configs, CI config, folder structure, 20 most recently changed files, and test files.
- Write `.athel/repos/<repo>/conventions.md`: language and framework versions, layering, naming, error handling, logging and metrics style, test framework and patterns (folder layout, naming, fixtures, mocking style, assertion style, how integration tests reach databases and other services), how to run build, tests and lint locally.
- Write `.athel/repos/<repo>/map.md`: modules and what they do, entry points, external clients, data access layer.
- Both are facts with recipes (doc 03): re-verified when the repo's build or lint config changes or after 30 days.
- The developer and reviewer agents must load `conventions.md` for every task in that repo.

New projects: with no code to learn from, the architect writes `conventions.md` during the spec step from the PRD's stack and the org's existing repos in the same language (if any), and confirms it with one decision card before any code is written.

Blast radius analysis

Blast radius is computed twice: from the spec (planned impact) before plan approval, and from the real diff (actual impact) before each PR; any difference between them blocks the PR until the plan is updated.

Analysis steps

1. **Change surface:** list what changes. From the spec: interfaces, events, schemas, configs. From the diff: changed public symbols, routes, message types, migrations, config keys, dependency versions.

2. **Contract classification:** for each interface change, classify as additive, behavioural or breaking (removed or renamed field, changed type, changed semantics, stricter validation, changed status codes, changed ordering or timing guarantees).
3. **Consumers:** for each changed contract, the scout answers `endpoint_callers` or `event_consumers` (doc 03), with traffic per environment.
4. **Data:** for each migration, table size, lock behaviour, backfill duration estimate, reversibility.
5. **Infra and config:** new env vars, secrets (names), resources, ingress or mesh changes, scaling needs.
6. **People:** owners of every affected service, active work overlapping, on-call for affected services.
7. **Runtime health:** current error rate, latency and alerts of every affected service; a change to an already unhealthy service raises risk.
8. **Score** with the rules below and write the report.

Risk rules (deterministic; the highest level that fires wins)

Level	Fires when any of these hold
CRITICAL	Breaking change to a contract with production consumers outside the change set; destructive migration (drop, rename, type narrowing) on a table with data; change touches auth, payments or PII handling without a security review task; affected service has a firing critical alert
HIGH	Breaking change where all consumers are in the change set; migration on a table over 1M rows; more than 3 services changed; new external dependency; change to retry, timeout or idempotency behaviour on a hot path (over 100 calls per second)
MEDIUM	Behavioural change to a contract with consumers; any migration; new config or secret; 2 to 3 services changed
LOW	Additive or internal change only, no migration, one service, runtime evidence confirms no unexpected consumers

Rules live in `policies/defaults/blast.yaml` and are overridable per org. The level decides extra gates: HIGH needs a rollback plan task; CRITICAL needs explicit human sign-off on the report itself in addition to plan approval.

Report (`blast-radius.md`, abridged)

```
# Blast radius · CHG-042 · planned · risk HIGH
Rules fired: breaking-in-set (phone → phone_number), migration > 1M rows

| Impact | Item | Evidence | Traffic (prod, 7d) |
|---|---|---|---|
| Changed | user-service PUT /v1/users/{id}/phone (new) | spec REQ-1 | n/a |
| Consumer | notification-service reads UserUpdated | F-0201 | 1,240
events/min |
| Consumer | crm-sync reads UserUpdated | F-0193 | 1,240 events/min |
| Consumer | admin-dashboard GET /v1/users/{id} | traces | 3 calls/day |
| Data | users.phone_number add + backfill | migration plan | 2.1M rows,
online |
| People | teammate PR user-service#812 overlaps src/profile/ | gh | active |

Not affected (checked): order-service (no reads of phone; code + traces)
Gaps: Coralogix unavailable; log evidence missing
```

The "Not affected (checked)" list is required: it shows what was ruled out and how, which is what reviewers need to trust the report.

Decision cards and when to ask

Athel decides alone when a choice is reversible, low-risk and covered by a rule; it asks when any of the ask rules fire, and it always asks with a recommendation.

Ask rules (any one fires a card)

Rule	Fires when
A1 Product meaning	Two or more reasonable readings of the PRD lead to different user-visible behaviour
A2 Irreversible	The choice is hard to undo: public contract shape, data model, migration, external dependency
A3 Risk	Blast radius is HIGH or CRITICAL, or the choice raises the risk level
A4 Conflict	PRD conflicts with existing behaviour, a lesson, a policy or another team's active work

Rule	Fires when
A5 Ownership	The change needs another team's service or approval
A6 Budget	Budget or retry limits reached
A7 Low confidence	The decision depends on a claim that is only `unverified` or `disputed`

Decide alone (no card) when all hold: reversible within the change; consistent with conventions, lessons and policy; not user-visible, or user-visible but specified; confidence `confirmed` or `likely`. The choice is still recorded in `decisions.yaml` as `decided_by: athel` with its reason, so reviewers can challenge it.

Card format (`decisions.yaml`, one entry)

```
- id: D-1
  rule: A1
  step: spec
  question: Which phone formats should the API accept?
  context: PRD-2.1 says "valid phone number". Current code accepts free text.
14% of stored numbers are not E.164 (scout, F-0210).
  options:
    - key: a
      label: E.164 only
      consequence: Clean data; mobile app must format before sending.
    - key: b
      label: E.164 plus local formats normalised at the API
      consequence: Friendlier; parser adds a dependency and edge cases.
  recommendation: {option: b, reason: "App already sends local formats
(traces show 61% non-E.164 requests)."}
  default_if_unanswered: b
  default_safe: true
  blocks: [T-03, T-04]
  answer: null            # filled by the human
  answered_by: null
```

Presentation: cards for a step are batched into one prompt (Claude Code's question tool where available, plain text otherwise), shortest first, each answerable with one choice. Measures from the Master PRD apply: median 2 cards per change, and every card has a "should not have asked" feedback option that tunes the rules.

The change plan and approval

The plan is the contract between the human and Athel: every write Athel makes later must match one of its tasks (doc 04, rule 7), so the plan is precise about repos, paths, verbs and checks.

Plan format (`plan.yaml`, one task)

```
schema: athel/plan@1
change: CHG-042
hash: 7c41…                # computed by the helper CLI; approval binds to
it
tasks:
  - id: T-03
    title: Add PUT /v1/users/{id}/phone with E.164 normalisation
    repo: user-service
    paths: ["src/profile/**", "src/validation/phone.ts", "test/profile/**"]
    verbs: [edit, create, test, commit, push]
    covers: [AC-1, AC-2]
    depends_on: [T-02]        # T-02 adds the column
    size: M                   # S < 100 lines, M < 250, L < 400; larger must
split
    gates: [build, unit, lint, types, coverage, contract, security-scan,
review]
    risk_notes: "Hot path; keep p95 under 40 ms (current 22 ms)"
    rollback: "Feature flag phone_v2_endpoint off"
    env_actions: []           # shared actions only, e.g. a ticket comment;
never a deploy
```

Planning rules

1. One task per coherent, reviewable unit; target size M or smaller; L is allowed only with a reason.
2. Tests are part of the task that changes behaviour, not a separate later task.
3. Migrations, contract additions and consumer updates are separate tasks, ordered by expand and contract.
4. Every remote or shared action (ticket comment, chat message; never a deploy, which is always the developer's) is listed in `env_actions` so it is approved with the plan, not discovered later.
5. Risky tasks carry a rollback (feature flag, revert-safe migration, config toggle).
6. Tasks that could run in parallel are marked by having no dependency between them (used in phase "Later" for worktree parallelism).

Approval

- `/athel:plan` ends by showing a one-screen summary: tasks per repo in order, gates, env actions, risk level, open assumptions, estimated cost and time.
- The human approves, edits, or rejects with comments. Approval is recorded with the plan hash (Master TRD, workflow engine).
- Any later change to `plan.yaml` (by Athel or a human) voids approval; small changes caused by a UAT round are re-presented as a plan diff, not a whole new plan.